

SageMaker Debugger Profiling Report

SageMaker Debugger auto generated this report. You can generate similar reports on all supported training jobs. The report provides summary of training job, system resource usage statistics, framework metrics, rules summary, and detailed analysis from each rule. The graphs and tables are interactive.

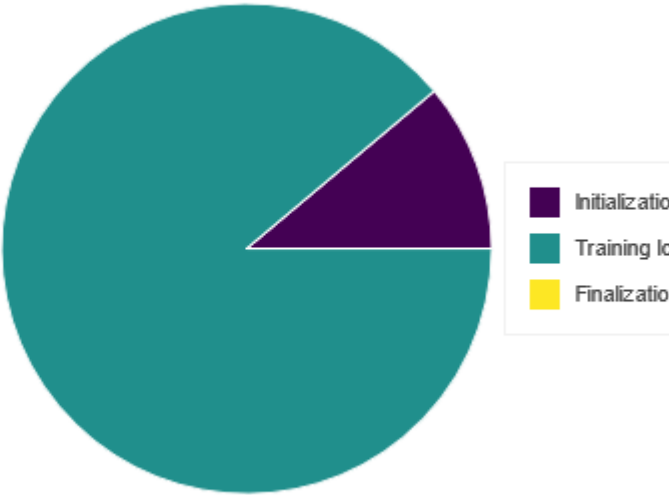
Legal disclaimer: This report and any recommendations are provided for informational purposes only and are not definitive. You are responsible for making your own independent assessment of the information.

```
In [4]: # Parameters
processing_job_arn = "arn:aws:sagemaker:us-east-1:559647231942:processing-job/pytorch-training-2025-01-1-ProfilerReport-c519bf98"
```

Training job summary

The following table gives a summary about the training job. The table includes information about when the training job started and ended, how much time initialization, training loop and finalization took. Your training job started on 01/14/2025 at 21:26:17 and ran for 2262 seconds.

#		Job Statistics
0	Start time	21:26:17 01/14/2025
1	End time	22:03:59 01/14/2025
2	Job duration	2262 seconds
3	Training loop start	21:30:28 01/14/2025
4	Training loop end	22:03:50 01/14/2025
5	Training loop duration	2001 seconds
6	Initialization time	250 seconds
7	Finalization time	9 seconds
8	Initialization	11 %
9	Training loop	88 %
10	Finalization	0 %



System usage statistics

The median total GPU utilization on node algo-1 is 0%. The median total CPU utilization is 35%.

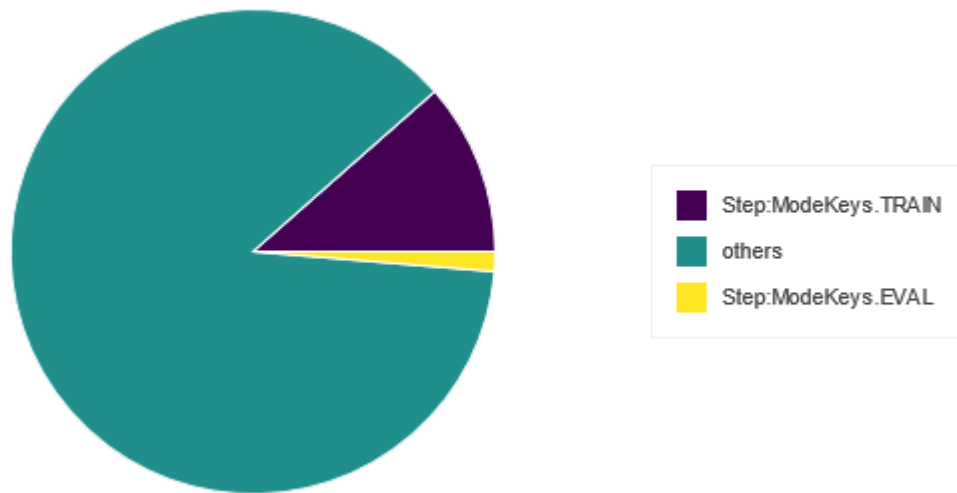
The following table shows statistics of resource utilization per worker (node), such as the total CPU and GPU utilization, and the memory utilization on CPU and GPU. The table also includes the total I/O wait time and the total amount of data sent or received in bytes. The table shows min and max values as well as p99, p90 and p50 percentiles.

#	node	metric	unit	max	p99	p95	p50	min
0	algo-1	Network	bytes	183596602.35	0	0	0	0
1	algo-1	GPU	percentage	86	86	78	0	0
2	algo-1	CPU	percentage	99.5	81.13	63.74	35.52	0
3	algo-1	CPU memory	percentage	36.33	33.16	31.97	31.45	4.43
4	algo-1	GPU memory	percentage	80	79	73	0	0
5	algo-1	I/O	percentage	33.91	24.4	14.69	0	0

Framework metrics summary

The following two pie charts show the time spent on the TRAIN phase, the EVAL phase, and others. The 'others' includes the time spent between steps (after or the next step has started). Ideally, most of the training time should be spent on the TRAIN and EVAL phases. If TRAIN/EVAL were not specified in the training set GLOBAL. Your training job spent quite a significant amount of time (87.2%) in phase "others". You should check what is happening in between the steps.

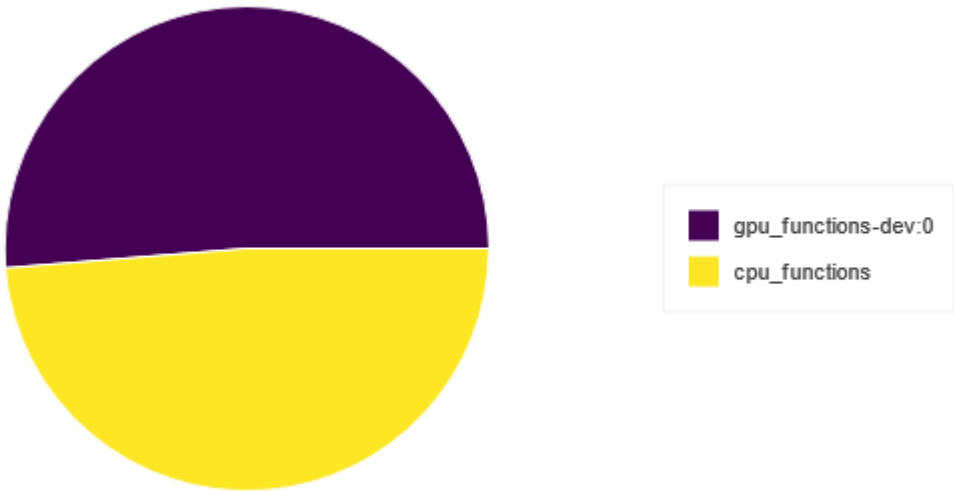
The ratio between the time spent on the TRAIN/EVAL phase and others



([http](http://boke)
boke

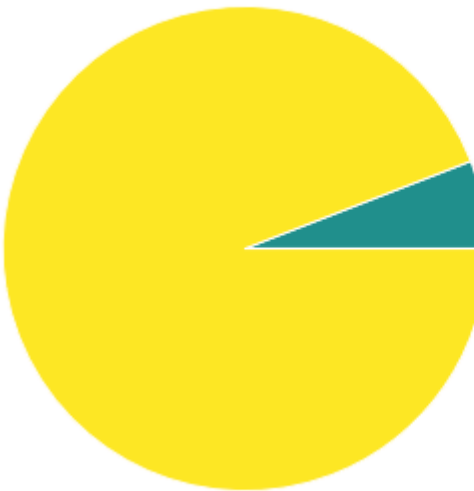
The following piechart shows a breakdown of the CPU/GPU operators. It shows that 51% of training time was spent on executing the "gpu_functions-dev:0" ope

The ratio between the time spent on CPU/GPU operators



General framework operations

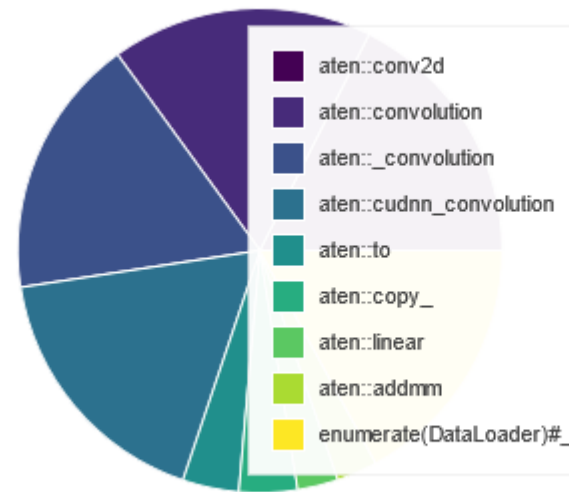
([http](#)
[boke](#)



Overview: CPU operators

The following table shows a list of operators that ran on the CPUs. The most expensive operator on the CPUs was "aten::conv2d" with 17.55%.

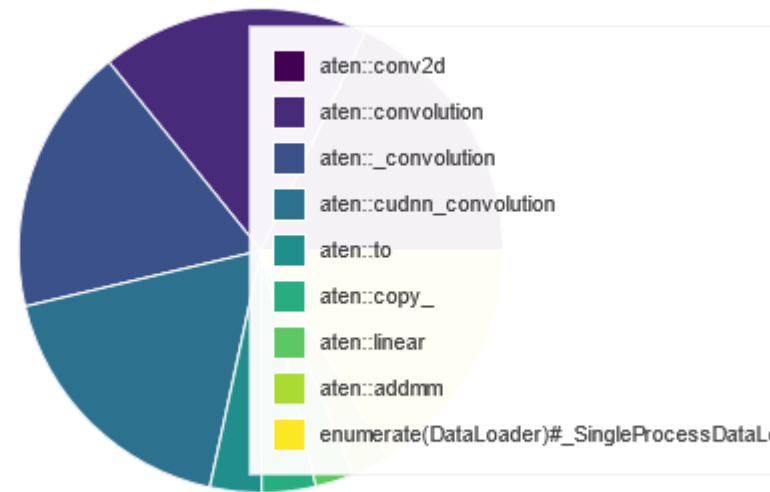
#	Percentage	Cumulative time in microseconds	CPU operator
0	17.55	8809768	aten::conv2d
1	17.46	8768815	aten::convolution
2	17.42	8748263	aten::_convolution
3	17.41	8740106	aten::cudnn_convolution
4	17.06	8564290	enumerate(DataLoader)#_SingleF
5	3.89	1954827	aten::copy_
6	3.76	1889503	aten::to
7	2.73	1370523	aten::linear
8	2.72	1363711	aten::addmm



Overview: GPU operators

The following table shows a list of operators that your training job ran on GPU. The most expensive operator on GPU was "aten::conv2d" with 17 %

#	Percentage	Cumulative time in microse	GPU operator
0	17.91	9457180	aten::conv2d
1	17.9	9454931	aten::convolution
2	17.9	9452980	aten::_convolution
3	17.89	9450580	aten::cudnn_convolution
4	16.22	8564394	enumerate(DataLoader)#_:
5	3.59	1895939	aten::copy_
6	3.46	1828952	aten::to
7	2.57	1354752	aten::linear
8	2.56	1354662	aten::addmm



Rules summary

The following table shows a profiling summary of the Debugger built-in rules. The table is sorted by the rules that triggered the most frequently. During your training job, the GPUMemoryIncrease rule was the most frequently triggered. It processed 4525 datapoints and was triggered 497 times.

	Description	Recommendation	Number of times rule triggered	Number of datapoints	Rule parameters
GPUMemoryIncrease	Measures the average GPU memory footprint and triggers if there is a large increase.	Choose a larger instance type with more memory if footprint is close to maximum available memory.	497	4525	increase:5 patience:1000 window:10
LowGPUUtilization	Checks if the GPU utilization is low or fluctuating. This can happen due to bottlenecks, blocking calls for synchronizations, or a small batch size.	Check if there are bottlenecks, minimize blocking calls, change distributed training strategy, or increase the batch size.	30	4525	threshold_p95:70 threshold_p5:10 window:500 patience:1000
StepOutlier	Detects outliers in step duration. The step duration for forward and backward pass should be roughly the same throughout the training. If there are significant outliers, it may indicate a system stall or bottleneck issues.	Check if there are any bottlenecks (CPU, I/O) correlated to the step outliers.	3	2159	threshold:3 mode:None n_outliers:10 stddev:3
LoadBalancing	Detects workload balancing issues across GPUs. Workload imbalance can occur in training jobs with data parallelism. The gradients are accumulated on a primary GPU, and this GPU might be overused with regard to other GPUs, resulting in reducing the efficiency of data parallelization.	Choose a different distributed training strategy or a different distributed training framework.	0	4525	threshold:0.2 patience:1000
Dataloader	Checks how many data loaders are running in parallel and whether the total number is equal the number of available CPU cores. The rule triggers if number is much smaller or larger than the number of available cores. If too small, it might lead to low GPU utilization. If too large, it might impact other compute intensive operations on CPU.	Change the number of data loader processes.	0	10	min_threshold:70 max_threshold:200
MaxInitializationTime	Checks if the time spent on initialization exceeds a threshold percent of the total training time. The rule waits until the first step of training loop starts. The initialization can take longer if downloading the entire dataset from Amazon S3 in File mode. The default threshold is 20 minutes.	Initialization takes too long. If using File mode, consider switching to Pipe mode in case you are using TensorFlow framework.	0	2159	threshold:20
CPUBottleneck	Checks if the CPU utilization is high and the GPU utilization is low. It might indicate CPU bottlenecks, where the GPUs are waiting for data to arrive from the CPUs. The rule evaluates the CPU and GPU utilization rates, and triggers the issue if the time spent on the CPU bottlenecks exceeds a threshold percent of the total training time. The default threshold is 50 percent.	Consider increasing the number of data loaders or applying data pre-fetching.	0	4528	threshold:50 cpu_threshold:90 gpu_threshold:10 patience:1000

	Description	Recommendation	Number of times rule triggered	Number of datapoints	Rule parameters
IOBottleneck	Checks if the data I/O wait time is high and the GPU utilization is low. It might indicate IO bottlenecks where GPU is waiting for data to arrive from storage. The rule evaluates the I/O and GPU utilization rates and triggers the issue if the time spent on the IO bottlenecks exceeds a threshold percent of the total training time. The default threshold is 50 percent.	Pre-fetch data or choose different file formats, such as binary formats that improve I/O performance.	0	4528	threshold:50 io_threshold:50 gpu_threshold:10 patience:1000
BatchSize	Checks if GPUs are underutilized because the batch size is too small. To detect this problem, the rule analyzes the average GPU memory footprint, the CPU and the GPU utilization.	The batch size is too small, and GPUs are underutilized. Consider running on a smaller instance type or increasing the batch size.	0	4524	cpu_threshold_p95:70 gpu_threshold_p95:70 gpu_memory_threshold_p95:70 patience:1000 window:500

Analyzing the training loop

Step duration analysis

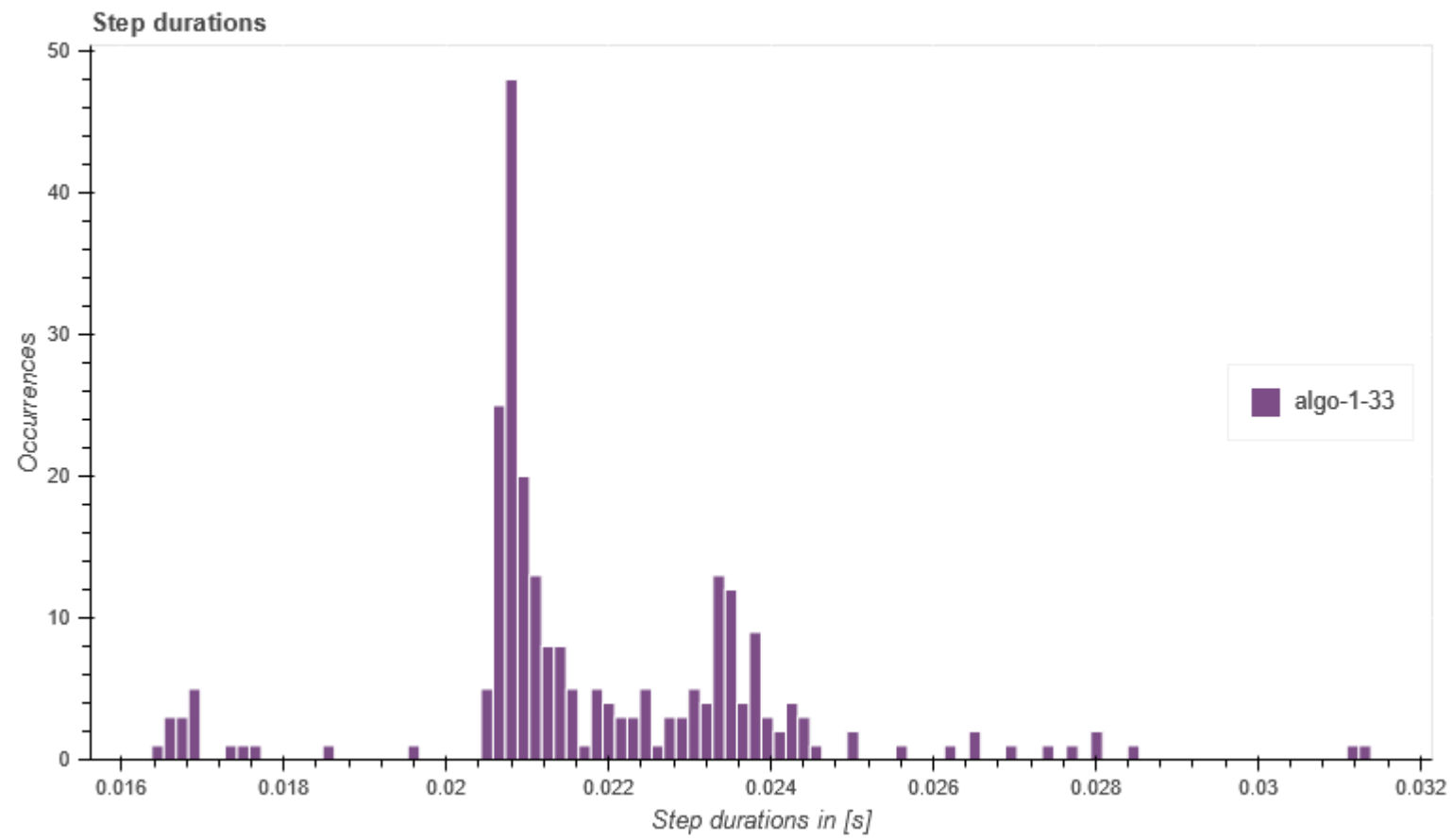
The StepOutlier rule measures step durations and checks for outliers. The rule returns True if duration is larger than 3 times the standard deviation. The rule also takes the parameter mode, that specifies whether steps from training or validation phase should be checked. In your processing job mode was specified as None. Typically the first step is taking significantly more time and to avoid the rule triggering immediately, one can use n_outliers to specify the number of outliers to ignore. n_outliers was set to 10. The rule analysed 2159 datapoints and triggered 3 times.

Step durations on node algo-1-33:

The following table is a summary of the statistics of step durations measured on node algo-1-33. The rule has analyzed the step duration from Step:ModeKeys.EVAL phase. The average step duration on node algo-1-33 was 0.06s. The rule detected 1 outliers, where step duration was larger than 3 times the standard deviation of 0.55s

	mean	max	p99	p95	p50	min
Step Durations in [s]	0.06	8.74	0.03	0.03	0.02	0.02

The following histogram shows the step durations measured on the different nodes. You can turn on or turn off the visualization of histograms by selecting or unselecting the labels in the legend.



(<http://bokeh.pydata.org/en/latest/docs/0.12.0/quickstart/interacting.html>)

GPU utilization analysis

Usage per GPU

The LowGPUUtilization rule checks for a low and fluctuating GPU usage. If the GPU usage is consistently low, it might be caused by bottlenecks or a small batch size. If usage is heavily fluctuating, it can be due to bottlenecks or blocking calls. The rule computed the 95th and 5th percentile of GPU utilization on 500 continuous datapoints and found 30 cases where p95 was above 70% and p5 was below 10%. If p95 is high and p5 is low, it might indicate that the GPU usage is highly fluctuating. If both values are very low, it would mean that the machine is underutilized. During initialization, the GPU usage is likely zero, so the rule skipped the first 1000 data points. The rule analysed 4525 datapoints and triggered 30 times.

Your training job is underutilizing the instance. You may want to consider to either switch to a smaller instance type or to increase the batch size. The last time that the LowGPUUtilization rule was triggered in your training job was on 01/14/2025 at 22:03:00. The following boxplots are a snapshot from the timestamps. They show the utilization per GPU (without outliers). To get a better understanding of the workloads throughout the whole training, you can check the workload histogram in the next section.

GPU utilization of gpu0 on node algo-1:

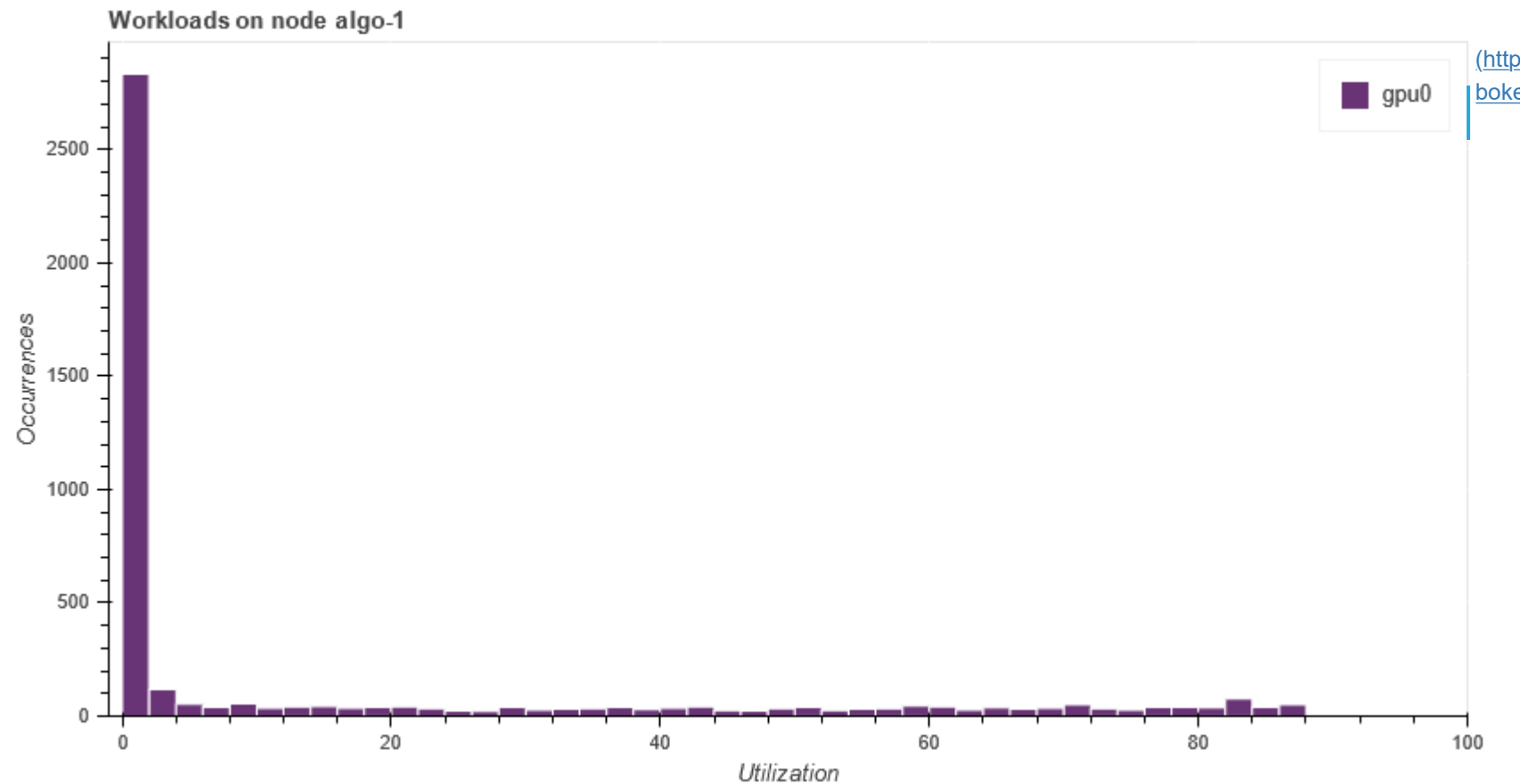
The max utilization of gpu0 on node algo-1 was 86.0% and the 5th percentile was only 0.0% The difference between 5th percentile 0.0% and 95th percentile 77.0% is quite significant, which means that utilization on gpu0 is fluctuating quite a lot.



Workload balancing

The LoadBalancing rule helps to detect issues in workload balancing between multiple GPUs. It computes a histogram of GPU utilization values for each GPU and compares then the similarity between histograms. The rule checked if the distance of histograms is larger than the threshold of 0.2. During initialization utilization is likely zero, so the rule skipped the first 1000 data points.

The following histogram shows the workload per GPU on node algo-1. You can enable/disable the visualization of a workload by clicking on the label in the legend. Your training job only used one GPU so there is no workload balancing issue.

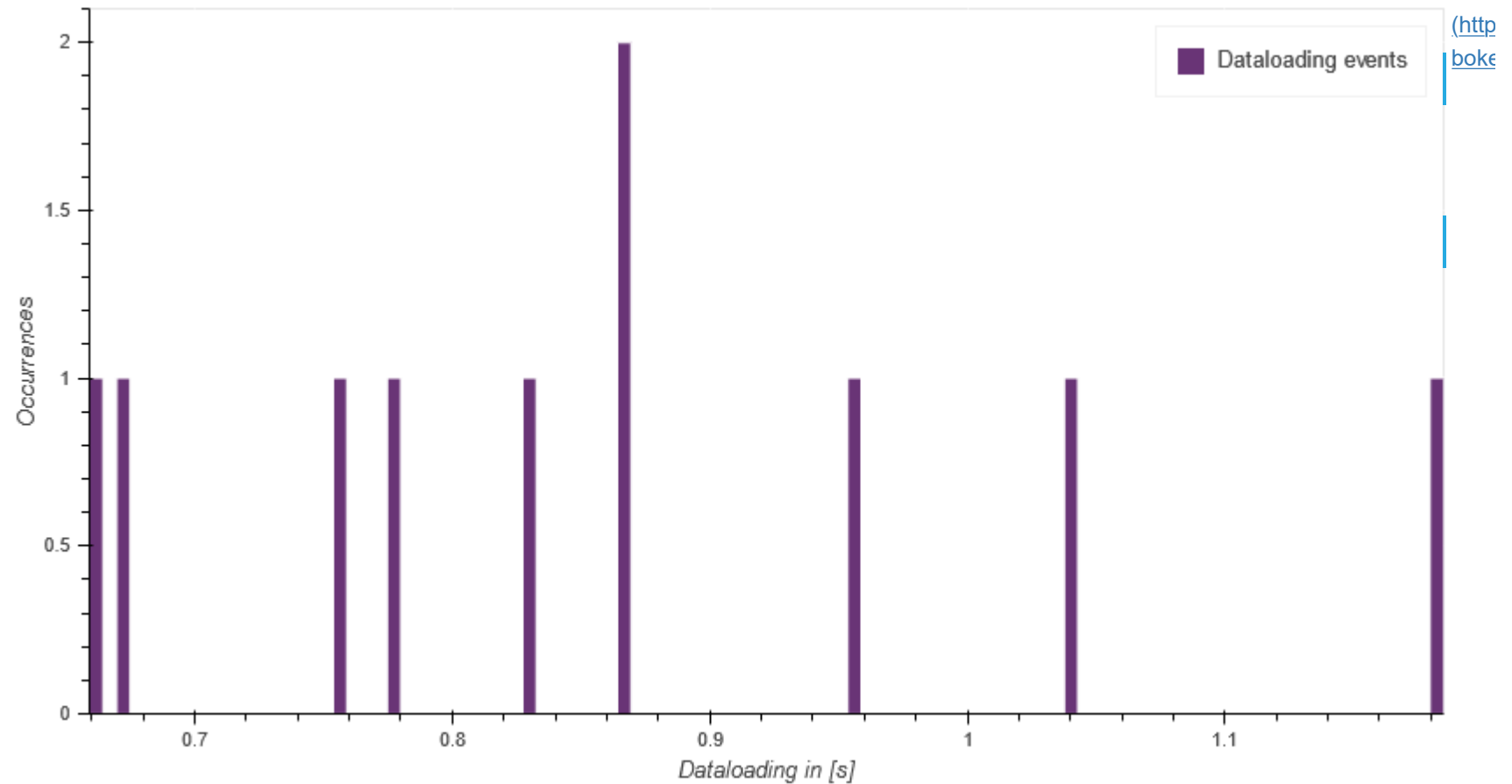


Dataloading analysis

The number of dataloader workers can greatly affect the overall performance of your training job. The rule analyzed the number of dataloading processes that have been running in parallel on the training instance and compares it against the total number of cores. The rule checked if the number of processes is smaller than 70% or larger than 200% the total number of cores. Having too few dataloader workers can slowdown data preprocessing and lead to GPU underutilization. Having too many dataloader workers may hurt the overall performance if you are running other compute intensive tasks on the CPU. The rule analysed 10 datapoints and triggered 0 times.

Your training instance provided 4 CPU cores, however your training job only ran on average 1 dataloader workers in parallel. We recommend you to increase the number of dataloader workers. Using pinned memory also improves performance because it enables fast data transfer to CUDA-enabled GPUs. The rule detected that your training job was not using pinned memory. In case of using PyTorch Dataloader, you can enable this by setting `pin_memory=True`

The following histogram shows the distribution of dataloading times that have been measured throughout your training job. The median dataloading time was 0.8482s. The 95th percentile was 1.1205s and the 25th percentile was 0.763s



Batch size

The BatchSize rule helps to detect if GPU is underutilized because of the batch size being too small. To detect this the rule analyzes the GPU memory footprint, CPU and GPU utilization. The rule checked if the 95th percentile of CPU utilization is below `cpu_threshold_p95` of 70%, the 95th percentile of GPU utilization is below `gpu_threshold_p95` of 70% and the 95th percentile of memory footprint below `gpu_memory_threshold_p95` of 70%. In your training job this happened 0 times. The rule skipped the first 1000 datapoints. The rule computed the percentiles over window size of 500 continuous datapoints. The rule analysed 4524 datapoints and triggered 0 times.

CPU bottlenecks

The CPUBottleneck rule checked when the CPU utilization was above `cpu_threshold` of 90% and GPU utilization was below `gpu_threshold` of 10%. During initialization utilization is likely to be zero, so the rule skipped the first 1000 datapoints. With this configuration the rule found 1938 CPU bottlenecks which is 42% of the total time. This is below the threshold of 50% The rule analysed 4528 data points and triggered 0 times.

I/O bottlenecks

The IOBottleneck rule checked when I/O wait time was above `io_threshold` of 50% and GPU utilization was below `gpu_threshold` of 10. During initialization utilization is likely to be zero, so the rule skipped the first 1000 datapoints. With this configuration the rule found 57 I/O bottlenecks which is 1% of the total time. This is below the threshold of 50%. The rule analysed 4528 datapoints and triggered 0 times.

GPU memory

The GPUMemoryIncrease rule helps to detect large increase in memory usage on GPUs. The rule checked if the moving average of memory increased by more than 5.0%. So if the moving average increased for instance from 10% to 16.0%, the rule would have triggered. During initialization utilization is likely 0, so the rule skipped the first 1000 datapoints. The moving average was computed on a window size of 10 continuous datapoints. The rule detected 497 violations where the moving average between previous and current time window increased by more than 5.0%. The rule analysed 4525 datapoints and triggered 497 times.

Your training job triggered memory spikes. The last time the GPUMemoryIncrease rule triggered in your training job was on 01/14/2025 at 22:03:00. The following boxplots are a snapshot from the timestamps. They show for each node and GPU the corresponding memory utilization (without outliers).

Memory utilization of gpu0 on node algo-1:

The max memory utilization of gpu0 on node algo-1 was 80.0%. The 5th percentile was only 0.0%. The difference between 5th percentile 0.0% and 95th percentile 73% is quite significant, which means that memory utilization on gpu0 is fluctuating quite a lot.

